Below is a **publication-quality end-to-end workflow** for your dataset that:

1. Trains an Autoencoder on invasive variables.
2. Learns latent microbiome representations.
3. Uses LOOCV (Leave-One-Out Cross Validation).
4. Predicts invasive variables from non-invasive variables.
5. Calculates:
   - Overall R^2
   - Overall RMSE
   - Per-genus R^2
   - Per-genus RMSE
   - Per-genus Pearson correlation
6. Exports results to Excel.

Imports

Python

Run

```python
import numpy as np
import pandas as pd

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import LeaveOneOut
from sklearn.metrics import r2_score
from sklearn.metrics import mean_squared_error

from scipy.stats import pearsonr

from tensorflow.keras.layers import Input
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout

from tensorflow.keras.models import Model

from tensorflow.keras.callbacks import EarlyStopping

from tensorflow.keras.regularizers import l2

import tensorflow as tf

np.random.seed(42)
tf.random.set_seed(42)
```

Load Dataset

Adjust the column ranges if needed.

Python

Run

```python
df = pd.read_excel(
    "Genus_Abundance_Fecal Data_Combined.xlsx"
)

# B:K
X = df.iloc[:,1:11].copy()

# L:BKK
Y = df.iloc[:,11:].copy()

# Missing values

X = X.fillna(X.mean())
Y = Y.fillna(Y.mean())

# Avoid log(0)

Y = Y.replace(0, 1e-8)

# Log transform

Y_log = np.log1p(Y)
```

Function: Autoencoder

Python

Run

```python
def build_autoencoder(
        input_dim,
        latent_dim=8
):

    encoder_input = Input(
        shape=(input_dim,)
    )

    x = Dense(
        256,
        activation='relu',
        kernel_regularizer=l2(0.001)
    )(encoder_input)
```

```
x = Dropout(0.5)(x)

x = Dense(
    64,
    activation='relu',
    kernel_regularizer=l2(0.001)
)(x)

latent = Dense(
    latent_dim,
    activation='linear'
)(x)

encoder = Model(
    encoder_input,
    latent
)

decoder_input = Input(
    shape=(latent_dim,)
)

x = Dense(
    64,
    activation='relu'
)(decoder_input)

x = Dense(
    256,
    activation='relu'
)(x)

decoder_output = Dense(
    input_dim,
    activation='linear'
)(x)

decoder = Model(
    decoder_input,
    decoder_output
)

auto_input = Input(
    shape=(input_dim,)
)

encoded = encoder(auto_input)

decoded = decoder(encoded)

autoencoder = Model(
    auto_input,
```

```
        decoded
    )

    autoencoder.compile(
        optimizer='adam',
        loss='huber'
    )

    return autoencoder, encoder, decoder
```

Function: Predictor

Python Run

```
def build_predictor(
        input_dim=10,
        latent_dim=8
):

    inp = Input(
        shape=(input_dim,)
    )

    x = Dense(
        16,
        activation='relu',
        kernel_regularizer=l2(0.001)
    )(inp)

    x = Dropout(0.5)(x)

    x = Dense(
        8,
        activation='relu',
        kernel_regularizer=l2(0.001)
    )(x)

    out = Dense(
        latent_dim,
        activation='linear'
    )(x)

    model = Model(inp, out)

    model.compile(
        optimizer='adam',
        loss='mse'
    )
```

```
    return model
```

LOOCV

This is the important part.

Python

```
loo = LeaveOneOut()

predictions = np.zeros(
    Y.shape
)

latent_dim = 8

sample_counter = 0

for train_idx, test_idx in loo.split(X):

    print(
        f"Sample {sample_counter+1}/39"
    )

    X_train = X.iloc[train_idx]
    X_test = X.iloc[test_idx]

    Y_train = Y_log.iloc[train_idx]
    Y_test = Y_log.iloc[test_idx]

    x_scaler = StandardScaler()
    y_scaler = StandardScaler()

    X_train_scaled = x_scaler.fit_transform(
        X_train
    )

    X_test_scaled = x_scaler.transform(
        X_test
    )

    Y_train_scaled = y_scaler.fit_transform(
        Y_train
    )

    # ----------------------
    # Autoencoder
    # ----------------------
```

```
autoencoder, encoder, decoder = \
    build_autoencoder(
        input_dim=Y_train.shape[1],
        latent_dim=latent_dim
    )

es = EarlyStopping(
    monitor='loss',
    patience=25,
    restore_best_weights=True
)

autoencoder.fit(
    Y_train_scaled,
    Y_train_scaled,
    epochs=300,
    batch_size=4,
    verbose=0,
    callbacks=[es]
)

latent_train = encoder.predict(
    Y_train_scaled,
    verbose=0
)

# ------------------------
# Predictor
# ------------------------

predictor = build_predictor(
    input_dim=10,
    latent_dim=latent_dim
)

predictor.fit(
    X_train_scaled,
    latent_train,
    epochs=300,
    batch_size=4,
    verbose=0,
    callbacks=[es]
)

latent_pred = predictor.predict(
    X_test_scaled,
    verbose=0
)

Y_pred_scaled = decoder.predict(
    latent_pred,
```

```
        verbose=0
    )

    Y_pred_log = y_scaler.inverse_transform(
        Y_pred_scaled
    )

    Y_pred = np.expm1(
        Y_pred_log
    )

    predictions[
        test_idx,
        :
    ] = Y_pred

    sample_counter += 1
```

Overall Metrics

Python

Run

```
true_values = Y.values

overall_r2 = r2_score(
    true_values,
    predictions,
    multioutput='uniform_average'
)

overall_rmse = np.sqrt(
    mean_squared_error(
        true_values,
        predictions
    )
)

print()
print("Overall Results")
print("----------------")
print("R2 =", overall_r2)
print("RMSE =", overall_rmse)
```

Per-Genus Metrics

Python

Run

```python
results = []

for i, genus in enumerate(Y.columns):

    y_true = true_values[:,i]

    y_pred = predictions[:,i]

    try:

        corr, p = pearsonr(
            y_true,
            y_pred
        )

    except:

        corr = np.nan

    r2 = r2_score(
        y_true,
        y_pred
    )

    rmse = np.sqrt(
        mean_squared_error(
            y_true,
            y_pred
        )
    )

    results.append(
        [
            genus,
            r2,
            rmse,
            corr
        ]
    )

metrics = pd.DataFrame(
    results,
    columns=[
        "Genus",
        "R2",
        "RMSE",
        "Pearson"
    ]
)
```

Top Predictable Genera

Python Run

```
print(
    metrics.sort_values(
        "R2",
        ascending=False
    ).head(20)
)
```

Export to Excel

Python Run

```
pred_df = pd.DataFrame(
    predictions,
    columns=Y.columns
)

with pd.ExcelWriter(
    "Microbiome_Prediction_Results.xlsx"
) as writer:

    metrics.to_excel(
        writer,
        sheet_name="Metrics",
        index=False
    )

    pred_df.to_excel(
        writer,
        sheet_name="Predictions",
        index=False
    )

print(
    "Results exported successfully."
)
```

A note on expectations

With only **39 subjects**, expect:

- Many genera to have **negative R^2**.
- Some dominant genera may show useful correlations.
- The latent dimension (`latent_dim`) is critical. Test values like **4, 8, 12, 16, 20** and compare LOOCV performance.
- Before modeling, it is often beneficial to remove genera that are present in very few samples (e.g., >90% zeros), as they are usually not predictable from anthropometric and dietary variables.

For a microbiome paper, I'd also suggest comparing this Autoencoder+Predictor model against:

- PLS Regression (baseline)
- MultiTask Elastic Net (baseline)
- Random Forest (baseline)

and reporting LOOCV results for all methods. This provides a much stronger scientific justification for the neural-network approach.